

Orchestrator state — Git-Vista dual-account parallel work

Mailbox location is fixed and absolute: `/home/tom/projects/Git-Vista/.claude/parallel/`. Both sessions read and write these files at that exact path regardless of which worktree they are working in. The directory is gitignored — it is live session state, not project history, and it deliberately does **not** travel through git. That means no commit/push/pull cycle is needed to exchange a message.

- **Milestone:** M1 — V1 Foundation (16 of 19 done)
- **Base SHA (both lanes branch from this):**
`635b3df232f4849b45d0bcab410855c35d1bc730`
- **Base description:** `main` immediately after PR #156 merged M1.11 (#64)
- **Orchestrator account:** Max (thomas2025)
- **Worker account:** Pro (thomas2010)

Standing architectural decisions (Max owns these; Pro does not change them)

- Frontend state lives in framework-free feature cores under `crates/git-vista/src/features/<name>/{core,signals}.rs`; `core.rs` is host-testable and has no `wasm32` dependency, `signals.rs` is `wasm`-only glue.
- At most one overlay per `Dock` — enforced structurally by `OverlayStack` (ADR 0024), not by convention at call sites.
- Writes go through the shared planner with typed plans (ADR 0015/0016) and one mutation at a time per repository (ADR 0019).
- The `.md` is always the record; its rendered PDF is a printable copy and lives only in `docs/superpowers/pdf/`, never beside its source.

Constraints that bind both lanes

- **Never** touch host port 8080 or restart the running git-vista server; Tom has a live iPad session on it. A server needed for verification binds an ephemeral port inside `systemd-run --user -p PrivateNetwork=yes`.
- **Never** delete a branch, local or remote. Standing rule, no exceptions.
- Commits use author `claude_2010` with `262510778+tom2025b@users.noreply.github.com`.
- One checkpoint per worktree, and it is that worktree's sole git writer. Worktrees have independent index and HEAD, so two checkpoints in two worktrees cannot race. Two sessions in the same directory would corrupt each other — hence worktrees are mandatory here, not optional.

Lane A — Max (this session)

Field	Value
Status	orchestrating; #66 M1.13 not yet started
Task	#66 M1.13 Centralize Git Process Execution Policy (Critical, security boundary)
Worktree	/home/tom/projects/Git-Vista (main checkout)
Branch	to be created: feature/m1.13-centralize-git-process-execution-policy
Owns these paths	crates/git-vista-server/**, crates/git-vista-git/**, crates/git-vista-protocol/**, docs/adr/0025-*

Lane B — Pro (worker)

Field	Value
Status	task 1 assigned, not started
Task	see pro-task.md
Worktree	/home/tom/projects/Git-Vista-pro
Branch	worker/pro/adr-0023-index-repair
Owns these paths	docs/adr/**, docs/superpowers/pdf/**

Why these two lanes cannot collide

Lane A works only inside `crates/**` plus one new ADR file it will create (`0025-*`). Lane B works only inside `docs/adr/**` and `docs/superpowers/pdf/**`. The single shared file is `docs/adr/README.md` — and Lane A does not touch it until Lane B's work is merged, because Lane A's ADR does not exist yet. Recorded here so the collision is a decision, not an accident.

The worker checkpoints ITSELF — and more often as its budget drains

The orchestrator's 60-second checkpointer covers the **main checkout only**. The worker's worktree has its own index and is not covered by it, and must not be: the worker performs its own git commits, so an orchestrator-run checkpointer pointed at the worker's worktree would race the worker on that index. Separate worktrees are what make two writers safe — one writer per worktree is the actual rule.

So the worker is its own checkpointer, and every task file must say so:

- **Commit and push whenever a meaningful step lands** — a reproduction achieved, a hypothesis ruled out, a diagnosis written down. Not only at the end. `wip(#NN): <what changed>` is fine; a messy WIP branch is cheap and redone investigation is not.
- **Checkpoint more frequently as the budget drops**. Past roughly 70% used, commit after every substantive step. Past 85%, write findings into `pro-result.md` first and

commit that, before attempting anything further — a diagnosis that reaches the orchestrator is worth more than a fix that dies uncommitted.

- **A bucket can end mid-sentence.** Uncommitted work is the only work that gets lost, and the worker's Pro bucket is the scarcer of the two.

Note on account sizes: worker is Pro, orchestrator is Max. Their percentages are not comparable units — 76% remaining on Pro can be less absolute capacity than 27% on Max. Size tasks to finish inside the worker's bucket, and prefer tasks whose partial result is still worth having.

Assigning a task is THREE files, never one

Learned the hard way on task 2. `pro-task.md` was updated correctly, but `pro-result.md` still read "task 1 — **done**". Pro checked the mailbox, saw a completed result, and correctly concluded there was nothing new to do. The orchestrator's bug, not the worker's.

A mailbox where two files disagree is worse than no mailbox, because the worker has no way to tell which one is stale. So every assignment updates all three, in this order:

1. **pro-task.md** — overwrite with the new task. Title it `Pro task N - ...` so a stale read is self-evident.
2. **pro-result.md** — reset to an empty template for the new task. Never leave the previous task's result sitting in it.
3. **state.json** — `worker.status` back to `assigned`, `new task`, `new branch`, clear `last_commit` and `pr`.

Same rule in reverse for the worker: finishing means writing `pro-result.md` AND setting `worker.status` to `done`. One without the other leaves the mailbox inconsistent in the same way.

Human relay (added after setup)

Tom is acting as a live relay between the two sessions. This is the channel the file mailbox cannot provide: the files are durable and async, but a question that needs answering now goes through him verbally in seconds instead of waiting for a poll.

Division of labour between the two channels:

- **Mailbox files** — task assignment, results, evidence, anything that must survive a session dying. The durable record.
- **Tom** — course corrections, clarifying questions, "stop, the base moved", anything time-sensitive.

Neither replaces the other. A decision relayed verbally still gets written into the mailbox afterwards, or the next session to resume has no idea it happened.

Experiment log (what this run is actually measuring)

Event	Time
Mailbox created, base SHA settled	2026-07-27T19:37-04:00
Task 1 assigned to Pro	2026-07-27T19:37-04:00
Pro session launched (sonnet/medium)	2026-07-27T19:58-04:00
Task 1 result received	—
Task 1 integrated	—

Setup cost, measured: 21 minutes from "let's try this" to Pro actually running. That covers the worktree, the isolated config dir, the global-rules symlink, the four mailbox files, the launch doc, and writing the task specification itself. Most of it is one-time and would not be paid again for task 2; the recurring per-task cost is writing `pro-task.md` and reading `pro-result.md`.

What the warm-up is and is not measuring. Task 1 is deliberately too small to save wall-clock time — restoring one file is maybe 20 minutes of work, less than the setup it just cost. It is measuring whether the protocol holds: whether an isolated session with no shared context can take a written task, stay inside its boundary, and hand back something reviewable. The #66/#67 split is where real savings would show, and that is experiment 2.

Honest prediction, recorded before the result so it cannot be rationalised afterwards: task 1 nets negative on time. The setup plus review will exceed doing it inline. Worth it only if the protocol proves reusable.

Lane A progress (Max)

- **66 M1.13 design pass running as workflow**
wa4h3urvp : survey → design →

three refutation lenses (attacker, operator, verifier).

- No production code touched — Tom's instruction was to settle invariants and test strategy before implementation.
- Empirically established this session, and written to `/tmp/claude-1000/-home-tom-projects-Git-Vista/74d494cf-ac3e-4ae7-bd7e-f3451291e0f2/scratchpad/verified-facts.md`: `GIT_CONFIG_NOSYSTEM=1` does **not** suppress global config (a global `core.hooksPath` still fires); `GIT_CONFIG_GLOBAL=/dev/null` does; and command-line `-c core.hooksPath=` overrides every scope. The repo's remote is HTTPS, not SSH, so the push dependency is a credential helper in global config — not `SSH_AUTH_SOCK`.

Signed: thomas2025 · 2026-07-27T19:37:13-04:00